

Evolutionary Database Design

Incremental improvements to the database schema so that it can be continuously updated with changes, reflecting the customer's requirements.

What EDD is about?

Is **not** a database design pattern

Is about how to maintain a database when the system is continuously **evolving**.

EDD is a **design process**, not the design itself.

DBAs collaborate closely with developers

WHY? The **developer** knows what new functionality is needed & **DBA** has a global view of the data in the application and other surrounding applications.

All database artifacts are version controlled with application code in the same repository that's being used by everyone else.

WHY? There's only one place to look. Easy to audit if any problems occur. Easy to handle out-of-sync database.

All database changes are migrations

Previously... developers would change the database as per their need using schema editing tools and ad-hoc SQL for standing data, then DBA would compare the development database with production database and update the changes. Here, the purpose of the changes needs be understood again, by a different group of people.

Now... every change is written in a migration script which is version controlled together with application code changes.

All database changes are migrations....

Migration script-

- Writes SQL into a text file inside a migrations folder within the project's version control repository
- Add unique IDs to each migration (look up the highest currently used number in the migrations folder, and uses that number together with a description to name the file.)
- Track which migrations were applied and their sequence using a **ChangeLog** table.

All database changes are migrations....

```
select * from changelog;
```

24/24Ln. 1 Col. 25Lines: 1INSERTWRITABLE \n UTF8Delimiter: ;

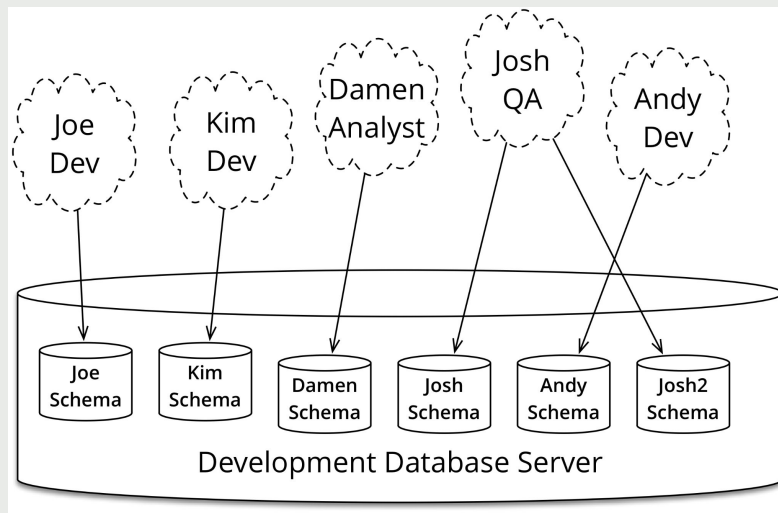
changelog

OFF

	CHANGE_NUMBER	COMPLETE_DT	APPLIED_BY	DESCRIPTION	
1	1	2016-08-04 12:12:13.475	jen_dev	1_asset.sql	
2	2	2016-08-04 12:12:13.525	jen_dev	2_asset_type.sql	
3	3	2016-08-04 12:12:13.57	jen_dev	3_asset_parameters.sql	
4	4	2016-08-04 12:12:13.586	jen_dev	4_inventory.sql	
5	5	2016-08-04 12:12:13.628	jen_dev	5_split_inventory_code.sql	
6	6	2016-08-04 12:12:13.671	jen_dev	6_equipment_type.sql	
7	7	2016-08-04 12:12:13.717	jen_dev	7_add_insurance_value_to_equipment_type.sql	
8	8	2016-08-04 12:12:13.723	jen_dev	8_data_location_equipment_type.sql	

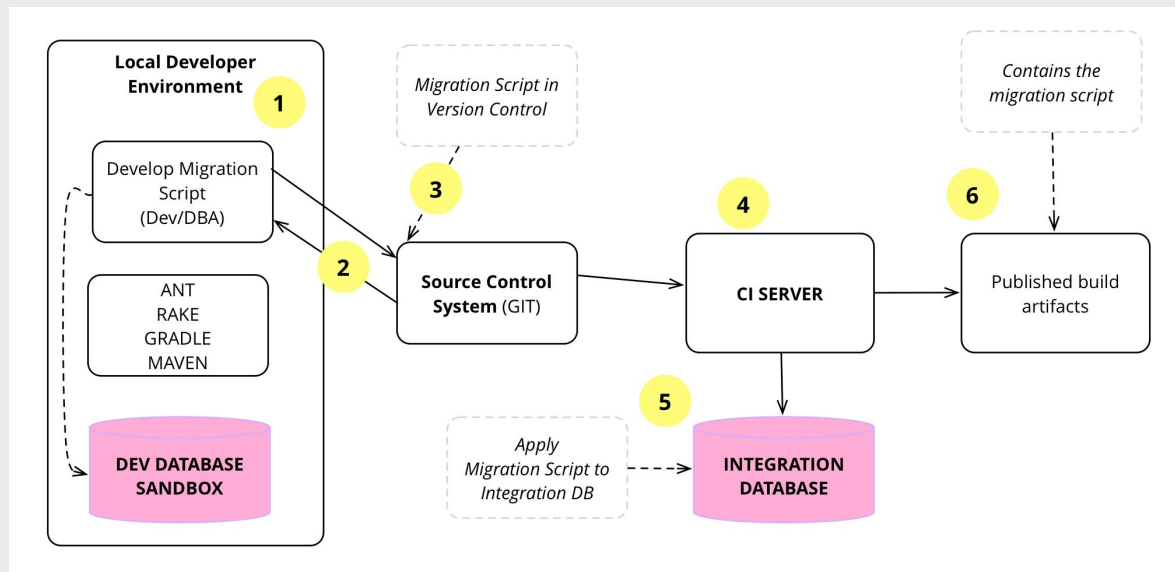
Everybody gets their own database instance

WHY? to enable isolated, uninterrupted experimentation—just like working with private code copies. This prevents conflicts from half-done changes in a shared database.



Developers continuously integrate database changes

WHY? to avoid complex, error-prone merges by catching and resolving conflicts early through frequent, small updates.



A database consists of schema and data

Not only code but also data- i.e. Static data (e.g. countries) or Sample data for testing (mostly fictional but can be real data from legacy system).

All database changes are refactorings

One of the big differences about database refactorings is that they involve **three** different changes that have to be done together-

- a. Changing the database schema
- b. Migrating the data in the database
- c. Changing the database access code

Thus whenever we describe a database refactoring, we have to describe all three aspects of the change and ensure that all three are applied before we apply any other refactorings.

Automate the refactorings

Frameworks like Liquibase and Active Record Migrations provide a DSL to apply database refactorings, allowing a standard way to apply database migrations.

Developers can update their database on-demand

Clearly separate all DB access code (separate database layer)

It reduces the need for developers to have deep SQL knowledge, simplifying their work. For DBAs, it offers a clear view of how the database is being modified, aiding in performance tuning and optimization.

Release frequently

Types of Database Changes

Every schema change falls into one of two categories: Non-destructive and Destructive. Destructive changes must be handled with a 3-phase approach.

Example A — Adding a column (Non-destructive)

User story: *"Users should be able to save their preferred language."*

```
ALTER TABLE users ADD COLUMN preferred_language VARCHAR(10) NULL;  
-- NULL means old rows are unaffected. Old code ignores the column. New code uses it.
```

Old code still works

Old data unaffected

New code opts in

Example B — Adding a new table (Non-destructive)

User story: *"Track which products are on a user's wishlist."*

```
CREATE TABLE wishlists (  
  id          SERIAL PRIMARY KEY,  
  user_id     INT REFERENCES users(id),  
  product_id  INT REFERENCES products(id),  
  added_at    TIMESTAMP DEFAULT NOW()  
);
```

Completely safe — no existing table is touched.

Destructive Changes

Example C — Renaming a column (Destructive → requires 3 phases)

User story: *"Rename phone to phone_number for clarity in our API."*

A direct rename breaks all existing queries, APIs, and UI components that use phone. EDD requires a **3-phase migration**:

Phase 1 — Start

Add the new column

Add phone_number as nullable.
Copy existing data. New code starts writing to both columns.

Phase 2 — Transition

Dual-write period

App writes to both phone and phone_number. Reads use COALESCE(phone_number, phone) as fallback.

Phase 3 — End

Safe deletion

Once all clients use phone_number, drop the old phone column.

```
-- Phase 1
ALTER TABLE users ADD COLUMN phone_number VARCHAR(20);
UPDATE users SET phone_number = phone;

-- Phase 2 (app reads)
SELECT COALESCE(phone_number, phone) AS phone_number FROM users;

-- Phase 3 (once all code is updated)
ALTER TABLE users DROP COLUMN phone;
```

User story: *"Users should be able to search by location, batch number, and serial number separately."*

Previously, inventory_code stored all three as a single string like CTG_023_768. The migration:

```
-- Step 1: Add the three new columns
ALTER TABLE inventory ADD COLUMN location_code VARCHAR(6) NULL;
ALTER TABLE inventory ADD COLUMN batch_number VARCHAR(6) NULL;
ALTER TABLE inventory ADD COLUMN serial_number VARCHAR(10) NULL;

-- Step 2: Migrate existing data by parsing the string
UPDATE inventory SET location_code = SUBSTR(inventory_code, 1, 6);
UPDATE inventory SET batch_number = SUBSTR(inventory_code, 8, 3);
UPDATE inventory SET serial_number = SUBSTR(inventory_code, 12, 3);

-- Step 3: Rebuild index on new columns, drop old column
DROP INDEX uidx_inventory_code;
CREATE UNIQUE INDEX uidx_inventory_identifier
  ON inventory (location_code, batch_number, serial_number);
ALTER TABLE inventory DROP COLUMN inventory_code;
```

This is a **database refactoring** — schema + data + access code all change together, in one commit.

User story: *"Each product can now have multiple categories, not just one."*

Currently: `products.category VARCHAR(50)`. The goal is a many-to-many join table.

```
-- Step 1: Create the lookup and join tables

CREATE TABLE categories (
  id SERIAL PRIMARY KEY,
  name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE product_categories (
  product_id INT REFERENCES products(id),
  category_id INT REFERENCES categories(id),
  PRIMARY KEY (product_id, category_id)
);

-- Step 2: Migrate existing category strings into the new tables

INSERT INTO categories (name)
  SELECT DISTINCT category FROM products WHERE category IS NOT NULL;

INSERT INTO product_categories (product_id, category_id)
  SELECT p.id, c.id FROM products p
  JOIN categories c ON c.name = p.category;

-- Step 3: (After all code is updated) drop old column

ALTER TABLE products DROP COLUMN category;
```

Example F — Adding a NOT NULL column (Tricky non-destructive case)

User story: *"Every order must now have a status field."*

Simply adding NOT NULL without a default will break *inserts* from old code that doesn't send the field. The safe EDD approach:

-- Step 1: Add with a DEFAULT first (non-breaking)

```
ALTER TABLE orders ADD COLUMN status VARCHAR(20) NOT NULL DEFAULT 'pending';
```

-- Step 2: Old code works unchanged (gets 'pending' automatically)

-- New code sends the actual status value

-- Step 3: (Optional later) remove the default if all code sets it explicitly

```
ALTER TABLE orders ALTER COLUMN status DROP DEFAULT;
```

Looks non-destructive...

...safe with a DEFAULT

How to do this?

4. Change any database code, such as views, stored procedures and triggers, to use the new columns
5. Change any indexes based on the `inventory_code` column
6. Modify/Run the test suits.
7. Commit this database migration script and all the application code changes to the version control system

References-

<https://martinfowler.com/articles/evodb.html>
